

# Experimentation protocol (working draft)

## Unsupervised structural clustering of the eukaryotic virome

Pascal Nardi — M2 internship

July 2, 2026

## Contents

|          |                                                         |          |
|----------|---------------------------------------------------------|----------|
| <b>1</b> | <b>General idea</b>                                     | <b>2</b> |
| <b>2</b> | <b>Dataset</b>                                          | <b>2</b> |
| 2.1      | Source . . . . .                                        | 2        |
| <b>3</b> | <b>Topological deep learning: from CNNs to CCNNs</b>    | <b>2</b> |
| 3.1      | Equivariance and invariance . . . . .                   | 3        |
| 3.2      | Convolutional networks: grids and translation . . . . . | 3        |
| 3.3      | Graph neural networks: graphs and permutation . . . . . | 3        |
| <b>4</b> | <b>Combinatorial complexes and CCNNs</b>                | <b>3</b> |
| 4.1      | Combinatorial complexes . . . . .                       | 4        |
| 4.2      | Neighborhood functions . . . . .                        | 4        |
| 4.3      | Cochains and higher-order message passing . . . . .     | 4        |
| 4.4      | Permutation equivariance . . . . .                      | 5        |
| <b>5</b> | <b>The Protein Combinatorial Complex</b>                | <b>6</b> |
| 5.1      | Neighborhood functions of the PCC . . . . .             | 6        |
| 5.2      | Featurization . . . . .                                 | 6        |
| 5.3      | Train / validation split . . . . .                      | 8        |
| <b>6</b> | <b>Architecture</b>                                     | <b>9</b> |
| 6.1      | Edge-centric frames and scalarization . . . . .         | 9        |
| 6.2      | The Topology-Complete Perceptron (TCP) . . . . .        | 9        |
| 6.3      | Embedding module . . . . .                              | 10       |
| 6.4      | Hierarchical message passing . . . . .                  | 10       |
| 6.5      | Modular ordering and the tensor-diagram flag . . . . .  | 11       |
| 6.6      | Readout and embedding . . . . .                         | 11       |
| 6.7      | Hyperparameters . . . . .                               | 12       |

|          |                                                   |           |
|----------|---------------------------------------------------|-----------|
| <b>7</b> | <b>Training method</b>                            | <b>12</b> |
| 7.1      | What contrastive learning is . . . . .            | 12        |
| 7.2      | Positive pairs by substructure sampling . . . . . | 13        |
| 7.3      | The InfoNCE objective . . . . .                   | 13        |
| <b>8</b> | <b>Plan of experiments</b>                        | <b>14</b> |
| 8.1      | Metrics . . . . .                                 | 14        |
| 8.2      | Baselines . . . . .                               | 16        |
| 8.3      | Design variables to sweep . . . . .               | 16        |
| 8.4      | Health checks I want to run every epoch . . . . . | 17        |

---

## 1 General idea

Viral proteins change in sequence much faster than they change in shape. Because of that, the usual sequence-based homology searches miss real relationships that are still visible at the level of the 3D fold. The idea behind DeltaFold is to learn, without labels, an embedding space where two viral proteins end up close together when they share a fold, so that clustering the embeddings recovers known protein families and possibly suggests structural links that sequence methods cannot find.

So the biological question I would like the method to be able to answer is whether an unsupervised structural encoder can recover viral protein families, and whether it can connect families that have diverged in sequence but are still structurally related. The methodological question I would like your opinion on is whether the setup below is enough to tell apart real structural learning from the model just exploiting easy shortcuts or collapsing its representation.

## 2 Dataset

### 2.1 Source

I use the predicted structural proteome of the eukaryotic virome from Nomburg et al. [1]. The release has 67,715 predicted protein structures across 4,463 eukaryotic viral species. The structures are organised by family on disk, which gives me weak family labels. I only use those labels to evaluate the model, never during training.

The dataset is developed further in later sections; the featurization of each structure and the train/validation split are deferred to Section 5.2, after the combinatorial-complex formalism they rely on.

## 3 Topological deep learning: from CNNs to CCNNs

This section gives the theoretical background that the architecture rests on. The goal is to explain why a topological model is a natural choice for protein structure, by following the line  $\text{CNN} \rightarrow \text{GNN} \rightarrow \text{CCNN}$  in which each step lifts a limitation of the previous one, and to make precise the notion of equivariance that organises this progression.

### 3.1 Equivariance and invariance

Let a group  $G$  act on an input space  $\mathcal{X}$  and on an output space  $\mathcal{Y}$ . A map  $f : \mathcal{X} \rightarrow \mathcal{Y}$  is  $G$ -equivariant if

$$f(g \cdot x) = g \cdot f(x) \quad \text{for all } g \in G, x \in \mathcal{X},$$

and  $G$ -invariant if  $f(g \cdot x) = f(x)$  for all  $g, x$ , the special case where  $G$  acts trivially on the output. Equivariance encodes a symmetry that the problem is known to have, so that the model does not need to learn it from data. The three families below are each defined by the symmetry group they respect, and each enlarges the domain on which the model can act.

### 3.2 Convolutional networks: grids and translation

A convolutional network acts on data supported on a regular grid (image pixels, audio samples). A convolutional layer slides one small filter across all positions; sharing the filter makes the layer equivariant to the translation group, that is, shifting the input shifts the output by the same amount. Weight sharing also gives locality and a parameter count independent of the input size. The limitation is that a grid is a rigid domain with a fixed neighbourhood and a global coordinate system, so CNNs do not apply to data whose elements have no canonical ordering or position, such as the residues of a protein whose contacts are irregular.

### 3.3 Graph neural networks: graphs and permutation

Graph neural networks remove the grid assumption. The data live on a graph  $G = (V, E)$  and a layer updates each vertex from its neighbours by message passing [4]:

$$h'_v = \beta\left(h_v, \bigoplus_{u \in \mathcal{N}(v)} \alpha(h_v, h_u)\right),$$

where  $\bigoplus$  is a permutation-invariant aggregation (sum, mean, or max). Because the aggregation ignores the order of the neighbours, a GNN is equivariant to permutations of the vertices: relabelling the nodes relabels the outputs the same way, which is the right symmetry for objects with no canonical node order. The limitation that motivates the next step is that a graph only encodes binary relations: an edge joins exactly two vertices. A protein, however, has natural higher-order parts, a whole  $\alpha$ -helix, a  $\beta$ -sheet, the entire chain, that are relations among many residues at once, and a plain graph has no first-class object for them. Reaching distant residues also requires stacking many layers, which tends to oversquash information through bottlenecks.

## 4 Combinatorial complexes and CCNNs

Combinatorial complex neural networks [3] keep permutation equivariance but replace the graph by a combinatorial complex, a domain with cells of several ranks: rank-0 cells are the vertices, rank-1 cells the edges, and higher-rank cells are groups of vertices treated as single objects, ordered by a rank that encodes a hierarchy. Message passing then happens not only between vertices but between cells of different ranks, so a helix (a rank-2 cell) can send and receive messages as a unit. This addresses both GNN limitations at once: higher-order parts become explicit objects, and a message between a residue and the helix it belongs to is a single step rather than a long chain of vertex-to-vertex hops. This section makes combinatorial complexes and their message passing precise.

## 4.1 Combinatorial complexes

**Definition 1** (Combinatorial complex, Hajij et al. [3]). A combinatorial complex is a triple  $(S, \mathcal{X}, \text{rk})$  where  $S$  is a finite set of entities,  $\mathcal{X} \subseteq \mathcal{P}(S) \setminus \{\emptyset\}$  is a set of cells, and  $\text{rk} : \mathcal{X} \rightarrow \mathbb{Z}_{\geq 0}$  is a rank function such that (i)  $\{s\} \in \mathcal{X}$  for every  $s \in S$ , and (ii)  $\text{rk}$  is monotone,  $x \subseteq y \Rightarrow \text{rk}(x) \leq \text{rk}(y)$ .

Following Hajij et al. [3], elements of  $S$  are called *entities* (or vertices) and elements of  $\mathcal{X}$  are *cells* (or relations); we take the convention  $\text{rk}(\{s\}) = 0$ . A cell  $x$  with  $\text{rk}(x) = k$  is a  $k$ -cell, written  $x^k$ ; the set of  $k$ -cells is  $\mathcal{X}^k$ , and the *dimension*  $\dim(\mathcal{X})$  is the largest rank present. The rank function gives a *hierarchy of relations*, while the absence of constraints on  $\mathcal{X}$  gives *set-type relations* (Def. 7–8 of [3]); a combinatorial complex is the domain that has both. Graphs (cells of rank 0 and 1 only) and cell complexes (cells closed under taking faces) are special cases. This is exactly the structure needed to host residues, contacts, secondary-structure elements and the whole protein — cells of ranks 0 to 3 — inside one object.

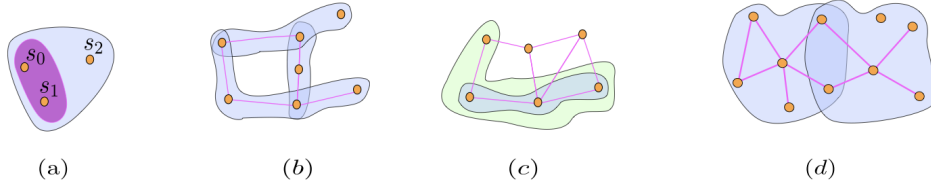


Figure 1: **Combinatorial complexes generalise graphs by allowing cells of several ranks.** Orange discs are vertices (rank-0 cells); pink, blue and green regions are cells of rank 1, 2 and 3, each a set of vertices treated as a single object. Examples (a), (b), (d) have dimension 2 and (c) has dimension 3. Adapted from Hajij et al. [3].

## 4.2 Neighborhood functions

A CC carries *neighborhood functions*  $\mathcal{N}$  (Def. 12 of [3]), which assign to a cell the set of cells in its local vicinity. Two are used here. Two cells are *incident* when one strictly contains the other; the down- and up-incidence functions of a cell  $x$  are

$$\mathcal{N}_{\downarrow}(x) = \{y \in \mathcal{X} \mid y \subsetneq x\}, \quad \mathcal{N}_{\uparrow}(x) = \{y \in \mathcal{X} \mid x \subsetneq y\},$$

linking cells of different ranks (a residue and the helix that contains it). Two same-rank cells are *adjacent* when they share a bridge cell of higher rank (Def. 17). A neighborhood function is stored as a binary neighborhood matrix; in particular the *incidence matrix*  $B_{r,k} \in \{0, 1\}^{|\mathcal{X}^r| \times |\mathcal{X}^k|}$  has  $[B_{r,k}]_{ij} = 1$  when the  $r$ -cell  $x_i^r$  is incident to the  $k$ -cell  $x_j^k$ . Each  $B_{r,k}$  induces a cochain map  $B_{r,k} : C^k(\mathcal{X}) \rightarrow C^r(\mathcal{X})$  (Def. 22), the discrete operator that moves signals between ranks.

## 4.3 Cochains and higher-order message passing

Data live on cells as *cochains*. The  $k$ -cochain space  $C^k(\mathcal{X}, \mathbb{R}^d)$  is the space of maps  $H_k : \mathcal{X}^k \rightarrow \mathbb{R}^d$  (Def. 21 of [3]); ordering the  $k$ -cells identifies it with  $\mathbb{R}^{|\mathcal{X}^k| \times d}$ , and we write  $H_k = [h_{x_1^k}, \dots]$  with  $H_{k,j} = h_{x_j^k}$  the feature of cell  $x_j^k$ . A CCNN layer updates the cochains by *higher-order message passing* over a set  $\mathcal{N} = \{\mathcal{N}_1, \dots, \mathcal{N}_n\}$  of neighborhood functions (Def. 34): for a cell  $x$  and a layer index  $l$ ,

$$m_{x,y} = \alpha_{\mathcal{N}_k}(h_x^{(l)}, h_y^{(l)}), \quad m_x^k = \bigoplus_{y \in \mathcal{N}_k(x)} m_{x,y}, \quad m_x = \bigotimes_{\mathcal{N}_k \in \mathcal{N}} m_x^k, \quad h_x^{(l+1)} = \beta(h_x^{(l)}, m_x),$$

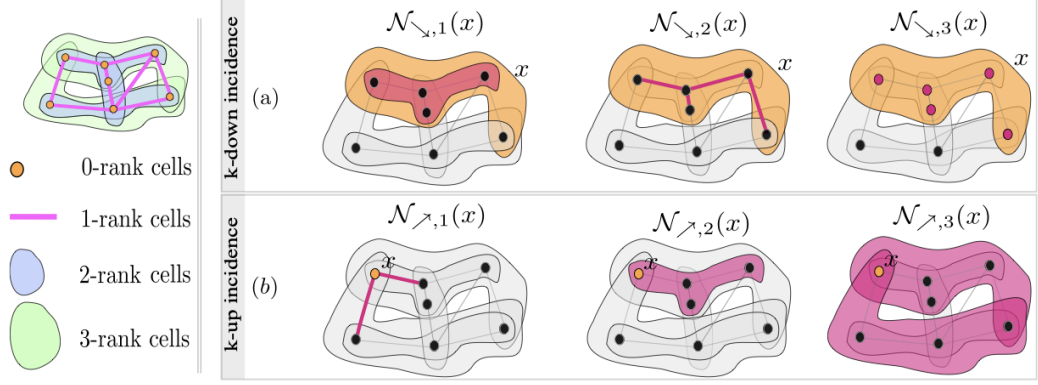


Figure 2: **Incidence neighborhood functions on a complex of dimension three** (cells coloured by rank, key at left). Top: the  $k$ -down incidences  $\mathcal{N}_{\searrow,1}, \mathcal{N}_{\searrow,2}, \mathcal{N}_{\searrow,3}$  of a target cell  $x$  — the cells one, two and three ranks below that  $x$  contains. Bottom: the  $k$ -up incidences  $\mathcal{N}_{\nearrow,k}$  of a vertex  $x$  — the higher-rank cells that contain it. Adapted from Hajij et al. [3].

where  $\oplus$  is a permutation-invariant *intra-neighborhood* aggregation,  $\otimes$  the *inter-neighborhood* aggregation over the chosen neighborhoods, and  $\alpha_{\mathcal{N}_k}, \beta$  are differentiable. When the messages are attention-weighted — a combinatorial complex attention network (CCANN, Def. 31–33) — the weight of neighbour  $y$  is  $\text{att}(x, y) = e_{xy} / \sum_{y' \in \mathcal{N}_k(x)} e_{xy'}$ , a distribution over  $\mathcal{N}_k(x)$ . This is the mechanism the model uses, with the neighborhoods given by the incidence functions of the PCC.

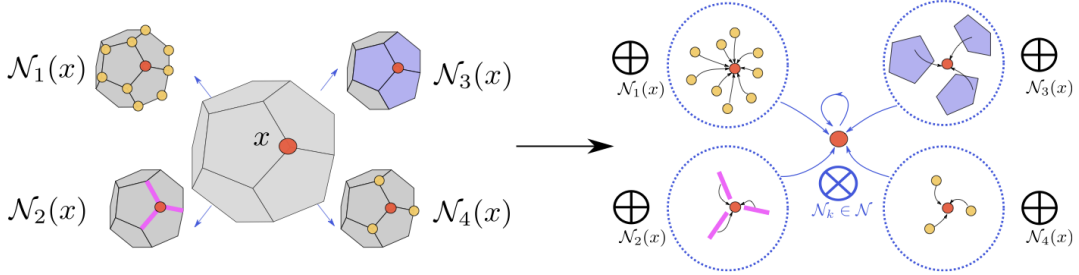


Figure 3: **One step of higher-order message passing (Def. 34)**. Left: for the target cell  $x$ , several neighborhood functions  $\mathcal{N}_1, \dots, \mathcal{N}_k$  are selected (different incidence or adjacency relations). Right: inside each neighborhood the incoming messages are combined by the intra-neighborhood aggregation  $\oplus$ ; the results from all neighborhoods are then merged by the inter-neighborhood aggregation  $\otimes$  and used to update  $x$ . Adapted from Hajij et al. [3].

#### 4.4 Permutation equivariance

Let  $P_k$  be a permutation matrix that relabels the  $k$ -cells. If every cochain is relabelled,  $H_k \mapsto P_k H_k$ , and every incidence matrix accordingly,  $B_{r,k} \mapsto P_r B_{r,k} P_k^\top$ , then a CCNN layer commutes with the relabelling, and its output transforms as  $H'_k \mapsto P_k H'_k$ . CCNNs are therefore permutation-equivariant, the higher-order analogue of the permutation equivariance of GNNs. A protein-level prediction is made permutation-invariant by ending with a symmetric pooling over cells.

## 5 The Protein Combinatorial Complex

Topotein [2] instantiates the framework of Section 4 for proteins. The Protein Combinatorial Complex (PCC) of a protein is a CC  $(S, \mathcal{X}, \text{rk})$  of dimension  $\dim(\mathcal{X}) = 3$  whose entities  $S$  are the residues, with cells of four ranks: the residues themselves ( $\mathcal{X}^0$ ), the spatial contacts ( $\mathcal{X}^1$ ), the secondary-structure elements (SSEs,  $\mathcal{X}^2$ ), and a single cell for the whole protein ( $\mathcal{X}^3$ ). The incidence functions used are those recorded by  $B_{0,1}$  (which residues form each contact),  $B_{0,2}$  (which residues compose each SSE), and  $B_{0,3}, B_{2,3}$  (that every residue and SSE lies in the protein cell).

Each rank carries two cochains, a scalar cochain and a vector cochain, transforming under two representations of  $SO(3)$ : the scalar  $\mathbf{s}_x \in \mathbb{R}^{n_r}$  under the trivial representation (it is *invariant*), the vector  $\mathbf{V}_x \in \mathbb{R}^{m_r \times 3}$  under the standard representation (it is *equivariant*,  $\mathbf{V}_x \rightarrow \mathbf{V}_x R^\top$  for  $R \in SO(3)$ ). The encoder (Section 6) preserves both transformation laws at every layer; the final readout is a scalar, hence  $SE(3)$ -invariant, and permutation-invariant after pooling — the property clustering requires.

### 5.1 Neighborhood functions of the PCC

Message passing on the PCC uses the incidence and adjacency neighborhoods of Section 4, in the directed form of Topotein [2]. Following their notation,  $B^{r \rightarrow r'} \in \{0, 1\}^{|\mathcal{X}^{r'}| \times |\mathcal{X}^r|}$  is the incidence matrix routing a rank- $r$  cochain to rank  $r'$ ; it is the  $B_{r,r'}$  (or its transpose) of Section 4 read as a directed map.

**Directed 1-cells.** Unlike the undirected hyperedges of the general combinatorial complex, the PCC’s 1-cells are *directed* pairwise edges. For an edge  $(i, j)$ , only the source  $i$  pushes to the edge (via  $B^{0 \rightarrow 1}$ ) and the edge pushes only to the target  $j$  (via  $B^{1 \rightarrow 0}$ ). This orientation is what makes the  $SO(3)$ -equivariant edge frames of Section 6 well defined.

**Outer-edge neighborhoods.** Each residue belongs to exactly one SSE, so the 2-cells never overlap and cannot communicate through ordinary down-adjacency (they would only echo messages back to themselves). To let SSEs exchange information, Topotein defines *outer-edge neighborhoods*

$$N_{\text{outer}}^{2 \rightarrow 1} = B^{2 \rightarrow 0} \cdot B^{0 \rightarrow 1} - B^{2 \rightarrow 1}, \quad (N_{\text{outer}}^{1 \rightarrow 2})^\top = B^{2 \rightarrow 0} \cdot (B^{1 \rightarrow 0})^\top - B^{2 \rightarrow 1},$$

where  $B^{2 \rightarrow 1}$  is the set of *inner* edges (both endpoints inside one SSE). Thus  $N_{\text{outer}}^{2 \rightarrow 1}$  maps an SSE to the edges that *start* inside it but *end* in a different SSE, and  $(N_{\text{outer}}^{1 \rightarrow 2})^\top$  to those that end inside it but start elsewhere: exactly the inter-SSE contacts, with one edge per contact so no geometric information is lost. These are the channels that carry tertiary packing (which strands pair, which helices pack) between secondary structures.

### 5.2 Featurization

Featurization (“lifting”) turns a predicted structure into a PCC, once per protein and offline. Only the  $C\alpha$  trace  $\mathbf{c}_1, \dots, \mathbf{c}_N \in \mathbb{R}^3$  of the first chain is used; every feature below is a function of these coordinates, so all scalar features are rigid-motion invariant and the sole raw vector feature is a displacement. Write  $N$  for the number of residues,  $k = 16$  the graph degree,  $E = kN$  the number of directed edges, and  $S$  the number of SSEs. For an edge  $e$ ,  $s(e)$  and  $t(e)$  are its source and target residues;  $\sigma(i) \in \{1, \dots, S\}$  is the SSE of residue  $i$ ; when proteins are batched,  $\beta(i)$  and  $\beta_2(p)$  map a residue or SSE to its protein. These maps are the incidence relations  $B_{0,1}, B_{0,2}, B_{0,3}, B_{2,3}$  read as functions.

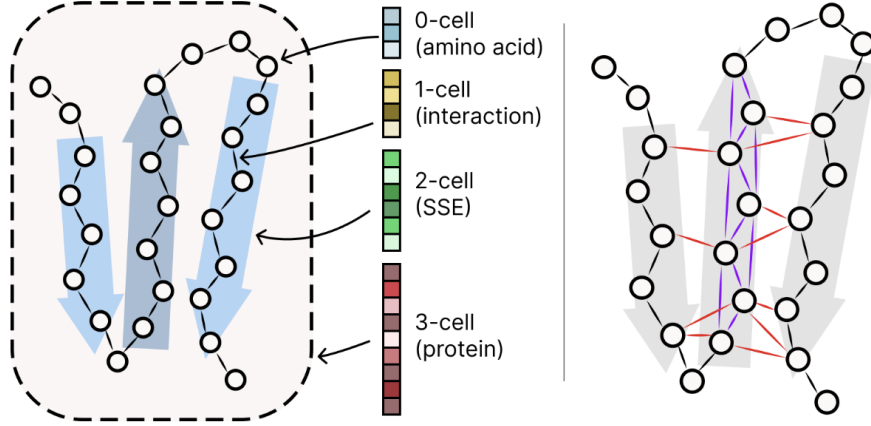


Figure 4: **The Protein Combinatorial Complex (PCC)**. Left: a protein as a four-rank complex — amino-acid nodes (0-cells, black circles), interaction edges (1-cells, black lines), secondary-structure elements (2-cells, blue arrows) and the whole protein (3-cell, dashed box); the colour key gives the rank of each cell. Right: inner edges (purple) lie inside a single SSE, whereas outer edges (red) connect two *different* SSEs — these are the contacts the outer-edge neighborhoods  $N^{2 \rightarrow 1}$  use to pass messages between secondary structures. Adapted from Toprotein [2].

**Rank 0 — residues.** The scalar feature is the backbone dihedral pair, sine/cosine encoded so the angle wraps smoothly:

$$\mathbf{s}_i^0 = (\sin \phi_i, \cos \phi_i, \sin \psi_i, \cos \psi_i) \in \mathbb{R}^4.$$

Residues carry no amino-acid identity, no 3Di token and no sequence-position encoding; the only other per-residue datum is the coordinate  $\mathbf{c}_i$ , which seeds the vector channels and the local frames of Section 6.

**Rank 1 — contacts.** A directed  $k$ -nearest-neighbour graph on the  $\text{C}\alpha$  atoms ( $k = 16$ , the self-neighbour excluded). Edge  $e$  from  $i = \text{s}(e)$  to  $j = \text{t}(e)$  carries a scalar and a vector,

$$d_e = \|\mathbf{c}_j - \mathbf{c}_i\|, \quad \mathbf{s}_e^1 = [d_e \parallel \text{RBF}(d_e)] \in \mathbb{R}^{1+K}, \quad \mathbf{v}_e^1 = \mathbf{c}_j - \mathbf{c}_i \in \mathbb{R}^{1 \times 3},$$

where  $\mathbf{v}_e^1$  is the only raw vector feature in the complex and  $\text{RBF}(\cdot)$  is a radial-basis expansion of the distance (specified in a later iteration).

**Ranks 2 and 3 — SSEs and protein.** Residues receive a DSSP-style three-state label (helix, strand, loop); maximal runs of one label form the SSEs, and  $\sigma(i)$  records membership. A point set  $\{\mathbf{c}_i\}_{i=1}^m$  (the residues of one SSE for rank 2, the whole chain for rank 3) with centroid  $\bar{\mathbf{c}}$  is summarised by the covariance and its spectrum,

$$\Sigma = \frac{1}{m-1} \sum_{i=1}^m (\mathbf{c}_i - \bar{\mathbf{c}})(\mathbf{c}_i - \bar{\mathbf{c}})^\top = \sum_{a=1}^3 \lambda_a \mathbf{u}_a \mathbf{u}_a^\top, \quad \lambda_1 \geq \lambda_2 \geq \lambda_3 \geq 0,$$

giving five scale-normalised shape descriptors

$$\left( \frac{\lambda_1 - \lambda_2}{\lambda_1}, \frac{2(\lambda_2 - \lambda_3)}{\lambda_1 + \lambda_2}, \frac{3\lambda_3}{\lambda_1 + \lambda_2 + \lambda_3}, \text{sign}(\prod_a \lambda_a) |\prod_a \lambda_a|^{1/3}, \frac{\lambda_1 - \lambda_3}{\lambda_1} \right).$$

The rank-2 scalar feature is the SSE type one-hot (4, with a catch-all slot), these five descriptors, and the three eigenvalues,  $\mathbf{s}_p^2 \in \mathbb{R}^{12}$ . The single rank-3 scalar feature stacks the size  $N$ , the radius

of gyration  $R_g = (\frac{1}{N} \sum_i \|\mathbf{c}_i - \bar{\mathbf{c}}\|^2)^{1/2}$ , and the five descriptors and three eigenvalues of the whole chain,  $\mathbf{s}^3 \in \mathbb{R}^{10}$ . All of these are functions of interatomic distances only, hence invariant to rotation and translation. Ranks 1–3 have no raw vector feature; their vector channels are seeded from  $\mathbf{v}^1$  in Section 6.3.

*visualisation placeholder — lifting one structure into the PCC:  $C\alpha$  trace  $\rightarrow k$ -NN contact graph (rank 1)  $\rightarrow$  DSSP segments (rank 2)  $\rightarrow$  protein cell (rank 3), with the scalar feature carried by each rank and the displacement vector on the edges.*

Figure 5: [to be added] Featurization of one protein into its Protein Combinatorial Complex.

### 5.3 Train / validation split

The split is cluster-aware and leakage-controlled, built once from the featurized pool. First, a small set of downstream control proteins is held out into a separate manifest and excluded from both train and validation. Second, the structures are grouped into their Foldseek structural clusters, and whole clusters are assigned entirely to train *or* to validation (a *cold* split), so no structural cluster — and hence no group of near-duplicate folds — spans the two sets. Finally, each cluster is subsampled at a common rate  $f = \text{target}/\text{pool} \approx 0.214$ , which scales every cluster (singletons included) by the same factor and so preserves the cluster-size distribution at the target size. The resulting split is disjoint at the level of structural clusters, and here yields 3,191 train and 813 validation proteins. Family labels are used only to evaluate the embeddings, never during training.

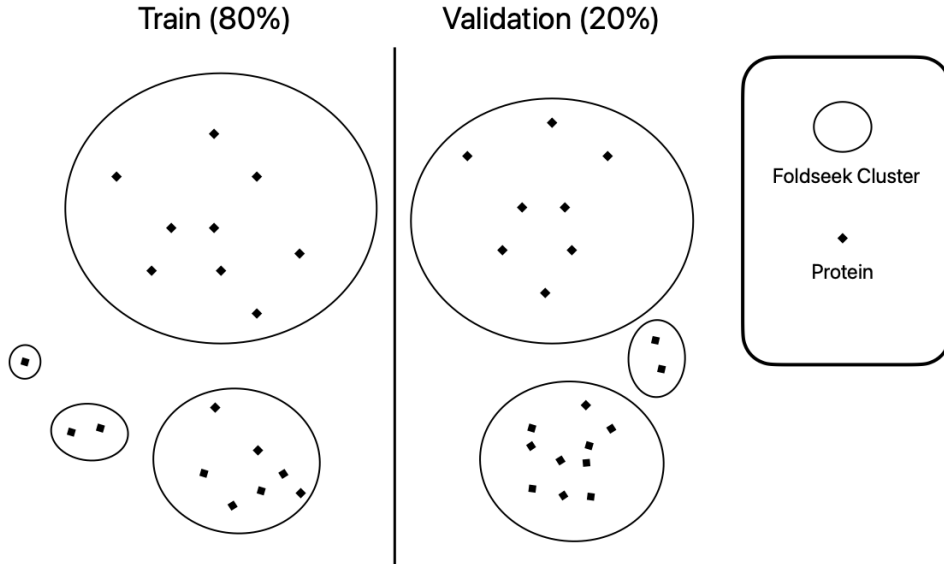


Figure 6: **Leakage-controlled cold cluster split.** Each ellipse is a Foldseek structural cluster and each diamond a protein. Whole clusters — not individual proteins — are assigned to train ( $\sim 80\%$ ) or validation ( $\sim 20\%$ ), so no structural cluster spans the split and no near-duplicate fold can leak from one set to the other. Each cluster is then subsampled at the common rate  $f$ , preserving the cluster-size distribution.

## 6 Architecture

The encoder is the Topology-Complete Perceptron Network (TCPNet) of Topotein [2]: an SE(3)-equivariant CCANN (Section 4) on the PCC. It processes scalar and vector cochains at every rank through the Topology-Complete Perceptron (TCP) module, with an embedding stage,  $L$  interaction layers of four-step hierarchical message passing, and a readout. We describe the full architecture faithfully; the one degree of freedom we add is that the *order* of the message-passing steps is made configurable through a tensor-diagram flag (Section 6.5). A batch holds  $B$  proteins; after embedding, the feature of a cell  $x^r$  is a pair  $(\mathbf{s}_x, \mathbf{V}_x)$  of scalar width  $d_s$  and vector width  $d_v$ , collected into the cochain  $H_r$ .

### 6.1 Edge-centric frames and scalarization

Vector features enter the invariant scalar stream only through projection onto local orthonormal frames. Topotein uses *edge-centric* frames: for the directed edge  $(i, j)$  with C $\alpha$  positions  $\mathbf{x}_i, \mathbf{x}_j$ ,

$$\mathbf{F}_{(i,j)} = \left( \frac{\mathbf{x}_j - \mathbf{x}_i}{\|\mathbf{x}_j - \mathbf{x}_i\|}, \frac{\mathbf{x}_j \times \mathbf{x}_i}{\|\mathbf{x}_j \times \mathbf{x}_i\|}, \frac{\mathbf{x}_j - \mathbf{x}_i}{\|\mathbf{x}_j - \mathbf{x}_i\|} \times \frac{\mathbf{x}_j \times \mathbf{x}_i}{\|\mathbf{x}_j \times \mathbf{x}_i\|} \right) \in \mathbb{R}^{3 \times 3},$$

translation-invariant and rotation-equivariant ( $\mathbf{F}_{(i,j)} \rightarrow \mathbf{F}_{(i,j)} R^\top$ ), with the cross-product axis flipping under reflection so the projection is chirality-sensitive. The scalarization  $S_i^{(r)}(\cdot)$  of a cell’s vector feature is the flattened projection onto the relevant frame(s), aggregated per rank:

$$\begin{aligned} \text{edge } (i, j) : \quad S_{(i,j)}(\mathbf{h}_{(i,j),v}^{(1)}) &= \text{flatten}(\mathbf{h}_{(i,j),v}^{(1)} \cdot \mathbf{F}_{(i,j)}), \\ \text{residue } i : \quad S_i(\mathbf{h}_{i,v}^{(0)}) &= \text{flatten}\left(\frac{1}{|B_i^{0 \rightarrow 1}|} \sum_{(i,j) \in B_i^{0 \rightarrow 1}} \mathbf{h}_{i,v}^{(0)} \cdot \mathbf{F}_{(i,j)}\right), \\ \text{SSE } i : \quad S_i(\mathbf{h}_{i,v}^{(2)}) &= \text{flatten}\left(\frac{1}{|N_i^{2 \rightarrow 1}|} \sum_{(l,j) \in N_i^{2 \rightarrow 1}} \mathbf{h}_{i,v}^{(2)} \cdot \mathbf{F}_{(l,j)}\right), \end{aligned}$$

i.e. residue vectors are read in the frames of their incident edges ( $B^{0 \rightarrow 1}$ ) and SSE vectors in the frames of their outer edges ( $N^{2 \rightarrow 1}$ , Section 5.1). The protein ( $r = 3$ ) frame is not edge-based — there are no inter-protein edges — but the three principal eigenvectors  $\mathbf{F}^{(3)} = (\mathbf{v}_1, \mathbf{v}_2, \mathbf{v}_3)$  of the C $\alpha$  point cloud, sign-disambiguated by the farthest-residue anchor of Bro et al. [12].

### 6.2 The Topology-Complete Perceptron (TCP)

The TCP module [2] is the rank-agnostic generalisation of the Geometry-Complete Perceptron [11]: it maps a  $(\mathbf{h}_s, \mathbf{h}_v)$  pair,  $\mathbf{h}_s \in \mathbb{R}^{d_s}$ ,  $\mathbf{h}_v \in \mathbb{R}^{d_v \times 3}$ , to updated scalar and vector features using the rank- $r$  frame. Two bias-free MLPs  $V_s, V_d$  reduce the vector channels, the reduced vectors are scalarized and concatenated with the scalars, and dedicated MLPs  $S_{\text{out}}, V_u$  with a scalar gate produce the outputs:

$$\begin{aligned} \mathbf{s} &= \sigma(V_s(\mathbf{h}_v)) \in \mathbb{R}^{3 \times 3}, & \mathbf{z} &= \sigma(V_d(\mathbf{h}_v)) \in \mathbb{R}^{(d_v/\lambda) \times 3}, \\ \mathbf{h}'_s &= (\mathbf{h}_s, S_i^{(r)}(\mathbf{s}), \|\mathbf{z}\|_2) \in \mathbb{R}^{d_s + 9 + d_v/\lambda}, & \mathbf{h}_{s,\text{out}} &= \sigma(S_{\text{out}}(\mathbf{h}'_s)) \in \mathbb{R}^{d_s}, \\ \mathbf{h}'_v &= \sigma(V_u(\mathbf{z})) \in \mathbb{R}^{d_v \times 3}, & \mathbf{h}_{v,\text{out}} &= \mathbf{h}'_v \odot \sigma_g(S_{\text{gate}}(\mathbf{h}_{s,\text{out}})), \end{aligned}$$

where  $\lambda$  is the bottleneck factor,  $\odot$  the element-wise product and  $\sigma_g$  a sigmoid gate that keeps the vector output equivariant while letting the scalars modulate its magnitude. The scalarization  $S_i^{(r)}$  is what lets a single module process residues, edges, SSEs and the protein cell uniformly while preserving SE(3)-equivariance. We write  $\text{TCP}^{(r)}$  for the module at rank  $r$ .

### 6.3 Embedding module

Raw features are first stabilised by GVP layer normalisation [10] (which normalises scalars and rescales vectors by their mean norm) and then mapped to the common width  $(d_s, d_v)$  by rank-specific TCP modules:

$$H^{(r)} = \varphi_{\text{emb}}^{(r)}(\text{LN}(H_0^{(r)})), \quad r \in \{0, 1, 2, 3\},$$

where  $H_0^{(r)}$  are the raw cochains of Section 5.2. Since the featurization stores a raw *vector* feature only on the edges ( $\mathbf{v}_e^1$ , the displacement), the residue, SSE and protein vector channels are seeded from it by mean aggregation up the ranks ( $\mathbf{v}_i^0 = \frac{1}{k} \sum_{e:s(e)=i} \mathbf{v}_e^1$ , then averaged into SSEs and the protein) before the embedding TCP.

### 6.4 Hierarchical message passing

Each interaction layer runs Topotein’s four-step scheme, a bottom-up-then-down sweep in which edges, SSEs, residues and the protein cell are updated in turn;  $\oplus$  denotes a permutation-invariant aggregation and every message/update function  $\phi^{(r)}$  is a TCP module (Section 6.2).

**Step 1 — edge messages.** Each edge integrates its source and target residues, its own feature, and the SSE features of both endpoints, then applies a TCP residual and a scalar-attention gate:

$$\mathbf{m}_{ij} = \phi^{(1)}(\mathbf{h}_i^{(0)}, \mathbf{h}_j^{(0)}, \mathbf{h}_{ij}^{(1)}, \mathbf{n}_i, \mathbf{n}_j), \quad \mathbf{n}_i = \begin{cases} \mathbf{h}_k^{(2)} & \exists k \in \mathcal{X}^2, i \in k \\ \mathbf{0} & \text{otherwise,} \end{cases}$$

$$\mathbf{h}_{ij}^{l+1} = \phi_{\text{msg}}(\mathbf{h}_{ij}^l) + \mathbf{h}_{ij}^l, \quad \mathbf{m}_{ij} = (\mathbf{h}_{ij,s}^{\text{fin}} \odot \sigma_{\text{att}}(\mathbf{w}^\top \mathbf{h}_{ij,s}^{\text{fin}}), \mathbf{h}_{ij,v}^{\text{fin}}).$$

**Step 2 — SSE update.** Each SSE aggregates its constituent residues ( $B^{2 \rightarrow 0}$ ), its inner edges ( $B^{2 \rightarrow 1}$ ) and, crucially, the edge messages of its outer-edge neighborhood ( $N^{2 \rightarrow 1}$ , the inter-SSE channel):

$$\mathbf{u}_i^{(2)} = \phi^{(2)}\left(\mathbf{h}_i^{(2)}, \bigoplus_{j \in B_i^{2 \rightarrow 0}} \mathbf{h}_j^{(0)}, \bigoplus_{j \in B_i^{2 \rightarrow 1}} \mathbf{h}_j^{(1)}, \bigoplus_{(j,k) \in N_i^{2 \rightarrow 1}} \mathbf{m}_{jk}\right).$$

**Step 3 — residue refinement.** Each residue is refined from its parent SSE and its incoming edge messages:

$$\mathbf{u}_i^{(0)} = \phi^{(0)}\left(\mathbf{h}_i^{(0)}, \mathbf{m}_i^{(2)}, \bigoplus_{j \in (B_i^{1 \rightarrow 0})^\top} \mathbf{m}_{ji}\right), \quad \mathbf{m}_i^{(2)} = \begin{cases} \mathbf{u}_k^{(2)} & \exists k \in \mathcal{X}^2, i \in k \\ \mathbf{0} & \text{otherwise.} \end{cases}$$

**Step 4 — protein update.** The global cell aggregates the updated residues and SSEs:

$$\mathbf{u}_i^{(3)} = \phi^{(3)}(\mathbf{u}_i^{(0)}, \mathbf{u}_i^{(2)}, \mathbf{h}_i^{(3)}).$$

**Feature update.** Ranks  $\{0, 2, 3\}$  are updated with a residual and layer normalisation (rank 1 is already updated in Step 1):

$$\mathbf{h}_i^{(r)'} = \text{LN}(\mathbf{u}_i^{(r)} + \mathbf{h}_i^{(r)}), \quad r \in \{0, 2, 3\}.$$

The stack applies  $L$  such layers. Because every attention weight is an invariant scalar and every function is a TCP, scalars stay SE(3)-invariant and vectors SO(3)-equivariant throughout. Note that, unlike the residue-hub simplifications, this scheme couples rank 1 and rank 2 directly: SSE features enter edge messages ( $\mathbf{n}_i, \mathbf{n}_j$  in Step 1) and edge messages update SSEs through the outer-edge channel (Step 2).

## 6.5 Modular ordering and the tensor-diagram flag

Each of the four steps is a cochain push-forward (Section 4) along a chosen neighborhood set, and a layer is their ordered composition. A CCNN is exactly specified by a *tensor diagram* (Def. 24 of [3]): a directed graph whose nodes are the cochains  $C^0 - C^3$  and whose labelled edges are the push-forwards ( $B^{0 \rightarrow 1}$ ,  $B^{2 \rightarrow 0}$ ,  $B^{2 \rightarrow 1}$ ,  $N^{2 \rightarrow 1}$ ,  $B^{1 \rightarrow 0}$ ,  $B^{0 \rightarrow 3}$ ,  $B^{2 \rightarrow 3}$ , and adjacencies such as  $A^{2 \sim 1}$ ), meeting at merge nodes where several messages reach the same target. Steps 1–4 above are one such diagram, and it is the default.

We make this diagram a *hyperparameter* of the training pipeline: the interaction layer is built from a declarative tensor-diagram specification passed as a flag (`--tensor-diagram`), an ordered list of (source ranks  $\rightarrow$  target rank, neighborhood) steps. The default reproduces TCPNet; alternative diagrams — reordering the steps, inserting a rank-2 adjacency  $A^{2 \sim 1}$  so SSEs talk through shared contacts, dropping the protein channel (replacing it by a pooling readout), or restricting to the residue hub as a control — are then swept without touching the model code (Section 8). The message-passing *order* thus becomes an experimental variable rather than a hard-coded choice.

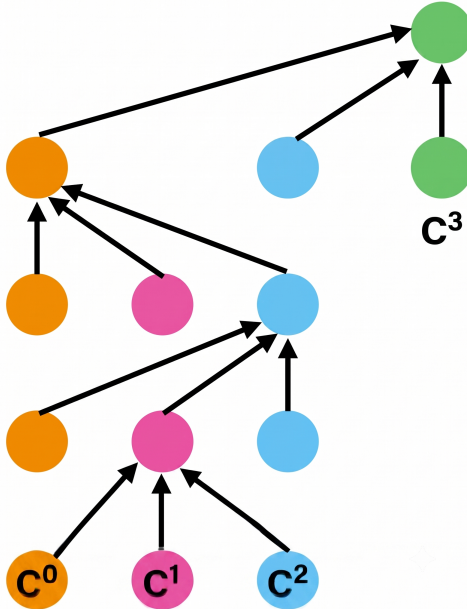


Figure 7: **Tensor diagram of one interaction layer (Topotein default).** The cochains  $C^0$  (residues, orange),  $C^1$  (edges, pink),  $C^2$  (SSEs, blue) and  $C^3$  (protein, green) enter at the bottom; each arrow is a push-forward along an incidence neighborhood and arrows meeting at a node are the merge that updates it. Reading upward: edges gather from residues, edges and SSEs ( $C^0, C^1, C^2 \rightarrow C^1$ ); SSEs update from residues, inner and outer edges ( $\rightarrow C^2$ ); residues refine from edges and their SSE ( $\rightarrow C^0$ ); the protein pools residues and SSEs ( $\rightarrow C^3$ ). This diagram is the object passed as the `--tensor-diagram` flag, so a different diagram gives a different message-passing order without changing the model code.

## 6.6 Readout and embedding

Topotein offers two readouts, and we treat the choice as a flag. Either (i) the final residue cochain  $H^{(0)}$  is pooled (sum or mean) into a protein vector — the rank-3 message passing is then unused — or (ii) the protein cochain  $H^{(3)}$  is used directly as the graph representation, which separates node- and graph-level capacity but is more prone to over-fitting. In both cases the resulting scalar representation  $\mathbf{r}_b \in \mathbb{R}^{d_s}$  is  $\ell_2$ -normalised to  $\mathbf{z}_b = \mathbf{r}_b / \|\mathbf{r}_b\| \in \mathbb{S}^{d_s-1}$  and used both for the contrastive loss (Section 7) and for evaluation. Since the scalar stream is SE(3)-invariant and the pooling

permutation-invariant,  $\mathbf{z}_b$  is invariant to rigid motion and residue reordering, and inverts under reflection.

## 6.7 Hyperparameters

| Group | Hyperparameter           | Value                                                     |
|-------|--------------------------|-----------------------------------------------------------|
| Model | encoder                  | TCPNet (TCP module, edge-centric frames)                  |
|       | scalar width $d_s$       | 128                                                       |
|       | vector width $d_v$       | 16                                                        |
|       | TCP bottleneck $\lambda$ | 3                                                         |
|       | interaction layers $L$   | 4                                                         |
|       | message-passing order    | Topotein 4-step (default; <code>--tensor-diagram</code> ) |
|       | readout                  | node pooling (default) or protein cell                    |
|       | dropout                  | 0.1                                                       |
|       | graph degree $k$         | 16                                                        |
|       | output dim               | 128, $\ell_2$ -normalised                                 |
| Optim | optimizer                | AdamW                                                     |
|       | learning rate            | $1 \times 10^{-4}$                                        |
|       | weight decay             | $1 \times 10^{-5}$                                        |
|       | scheduler                | reduce-on-plateau (factor 0.5, patience 2)                |
|       | gradient clip            | value clip 1.0                                            |
|       | epochs                   | 50                                                        |
|       | batch size               | 256 proteins                                              |
| Loss  | objective                | InfoNCE / NT-Xent (in-batch negatives)                    |
|       | temperature $\tau$       | 0.2                                                       |
|       | positives                | two substructures, fraction $f \in [0.5, 0.8]$            |

## 7 Training method

### 7.1 What contrastive learning is

The model is trained without any family labels, using contrastive learning. The principle is simple: I take each protein and produce two *views* of it, then I train the encoder so that the two views of the same protein land close together in the embedding space while views of different proteins are pushed apart. Over many proteins, the only way to satisfy this everywhere is to place proteins that are genuinely alike near each other and proteins that differ far apart. The encoder therefore discovers, on its own, the features that make two proteins similar, without ever being told what the families are. Two elements determine whether this works: how the two views are made (the positive pair) and where the negatives come from. The rest of this section specifies both.

There are two useful ways to read this objective. First, in information terms, the two views are two partial observations of the same underlying structure, and the contrastive loss is a lower bound on the mutual information between their embeddings [7]: maximising it forces the embedding to keep exactly what is common to both views (the fold) and to discard what differs between them (the particular region sampled). Second, on the unit sphere the loss splits into two competing forces [5]: an alignment force that pulls the two views of one protein together, and a uniformity force, carried by the negatives, that spreads all proteins out over the sphere. Without negatives the alignment force alone is minimised by mapping every protein to the same point, which is the representational collapse the health checks of Section 8.4 are meant to catch; the negatives are what keep the embedding spread out and informative.

## 7.2 Positive pairs by substructure sampling

The first attempt made the two views by adding small Gaussian coordinate noise and masking a few features, in the SimCLR style [6]. This failed in a specific and instructive way: the two views were almost identical, the model could match them by being invariant to noise alone, and the loss collapsed to about  $10^{-5}$  within one epoch without learning anything about the fold. An  $SE(3)$ -equivariant encoder is already invariant to rigid motion and largely insensitive to sub-Å jitter, so noise-based positives are close to trivial for it.

I therefore replaced noise augmentation with *substructure sampling* [15]: the positive pair is two distinct, connected substructures of the *same* protein. Concretely, each view keeps a contiguous segment of the chain (a random fraction  $f \in [0.5, 0.8]$  of the residues, floored at 17 so the  $k = 16$  graph stays valid) and the whole complex is re-lifted on that subset — the  $C\alpha$  graph, the secondary-structure elements and the global descriptors are all recomputed on the retained residues. Recognising that two different regions of a chain belong to the same fold is a genuinely non-trivial task that cannot be solved by noise-invariance, which is exactly what the first recipe lacked; the same idea underlies the sub-sequence/sub-space crops of GearNet [16]. The size fraction  $f$  is the critical knob (an information- bottleneck trade-off): too small and the two views no longer share the fold, too large and they become near-identical again.

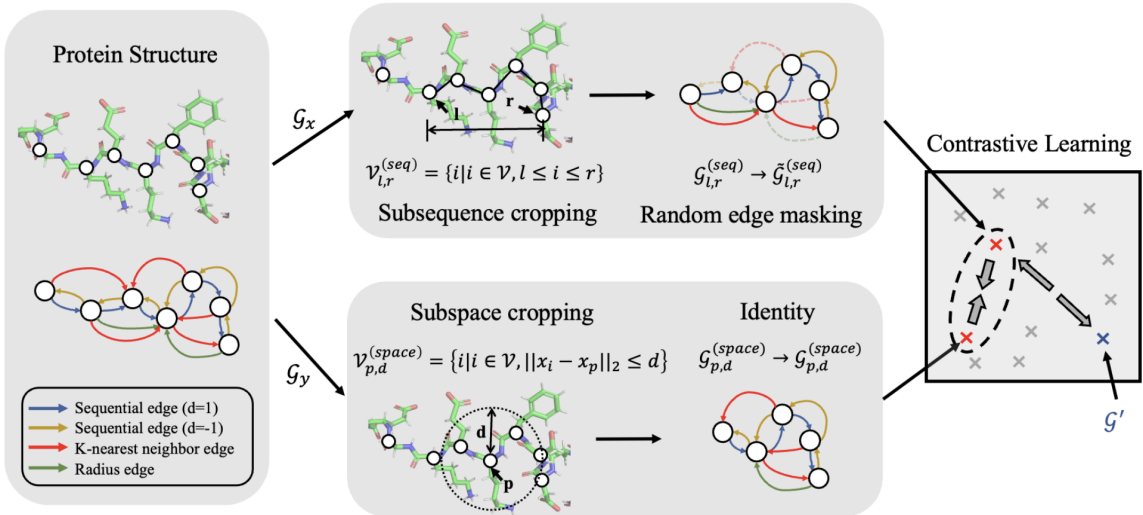


Figure 8: **Building the positive pair by substructure sampling.** From one protein’s residue graph, two views  $\mathcal{G}_x, \mathcal{G}_y$  are drawn by cropping a connected substructure: either a contiguous chain segment (subsequence, top) or a spatial ball around a residue (subspace, bottom), optionally followed by light edge masking. The contrastive loss pulls the two views together in latent space and pushes other proteins ( $\mathcal{G}'$ ) apart. Adapted from GearNet [16]; the substructure-crop idea follows Hermosilla and Ropinski [15].

## 7.3 The InfoNCE objective

The objective is the NT-Xent form of InfoNCE [7, 6], with negatives taken from the batch. A step samples  $M$  proteins and two substructure views of each, giving  $2M$  embeddings  $\mathbf{z}_1, \dots, \mathbf{z}_{2M} \in \mathbb{S}^{d_s-1}$  (Section 6.6); the two views of one protein form a positive pair  $(i, j)$ , and the other  $2M - 2$

views are its negatives. With cosine similarity  $\text{sim}(\mathbf{u}, \mathbf{v}) = \mathbf{u}^\top \mathbf{v}$  and temperature  $\tau$ ,

$$\ell_{i,j} = -\log \frac{\exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_j)/\tau)}{\sum_{a=1}^{2M} \mathbf{1}_{[a \neq i]} \exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_a)/\tau)}, \quad \mathcal{L} = \frac{1}{2M} \sum_{(i,j)} \ell_{i,j},$$

the sum running over both ordered directions of every positive pair, with  $\tau = 0.2$ . The loss acts directly on the sphere embedding  $\mathbf{z}$ : there is no projection head and no momentum encoder. A large batch (a single lab GPU allows a few hundred proteins, i.e. several hundred negatives per anchor) supplies enough negatives for the uniformity term of Section 7 to spread the embeddings, so no negative queue is needed.

## 8 Plan of experiments

### 8.1 Metrics

I evaluate the embeddings by clustering them (HDBSCAN [9], density-based) and comparing the result to the Foldseek clusters used as reference labels. A single overlap score such as the adjusted Rand index (ARI) tells me *whether* the two partitions agree but not *how* they disagree, and the two ways a clustering can be wrong are opposite and worth separating. I therefore report a set of directional agreement metrics as the primary readout, and keep ARI and TM- $\rho$  as secondary summaries.

**Notation.** Let  $\mathcal{C} = \{C_1, \dots, C_{|\mathcal{C}|}\}$  be the reference (Foldseek) clusters and  $\mathcal{K} = \{K_1, \dots, K_{|\mathcal{K}|}\}$  the clusters found in the embedding, over  $n$  proteins. Write  $n_{ck} = |C_c \cap K_k|$  for the contingency counts,  $n_c = |C_c|$ ,  $n_k = |K_k|$ , and denote entropies and conditional entropies by

$$H(\mathcal{C}) = -\sum_c \frac{n_c}{n} \log \frac{n_c}{n}, \quad H(\mathcal{C} \mid \mathcal{K}) = -\sum_k \frac{n_k}{n} \sum_c \frac{n_{ck}}{n_k} \log \frac{n_{ck}}{n_k},$$

and symmetrically  $H(\mathcal{K})$ ,  $H(\mathcal{K} \mid \mathcal{C})$ .

**Homogeneity, completeness, V-measure.** Following Rosenberg–Hirschberg [18],

$$h = 1 - \frac{H(\mathcal{C} \mid \mathcal{K})}{H(\mathcal{C})}, \quad c = 1 - \frac{H(\mathcal{K} \mid \mathcal{C})}{H(\mathcal{K})}, \quad V = \frac{2hc}{h+c}.$$

Homogeneity  $h$  is 1 when every learned cluster is pure in reference labels ( $H(\mathcal{C} \mid \mathcal{K}) = 0$ ); completeness  $c$  is 1 when every reference fold lands in a single learned cluster. They trade off — singletons maximise  $h$ , one giant cluster maximises  $c$  — and  $V$  is their harmonic mean; reporting  $h$  and  $c$  separately shows which side is failing.

**Fragmentation and fusion.** With  $\mathcal{K}(C_c) = \{k : n_{ck} > 0\}$  the set of learned clusters a reference fold touches, and  $\mathcal{C}(K_k) = \{c : n_{ck} > 0\}$  the reference folds a learned cluster touches,

$$\text{frag} = \frac{1}{|\mathcal{C}|} \sum_c |\mathcal{K}(C_c)|, \quad \text{fus} = \frac{1}{|\mathcal{K}|} \sum_k |\mathcal{C}(K_k)|.$$

Fragmentation is the mean number of learned clusters a fold is split across (the cluster-level face of low completeness); fusion is the mean number of folds merged into one learned cluster (the face of low homogeneity); both are 1 when ideal. High frag with low fus means over-splitting; the reverse means over-merging.

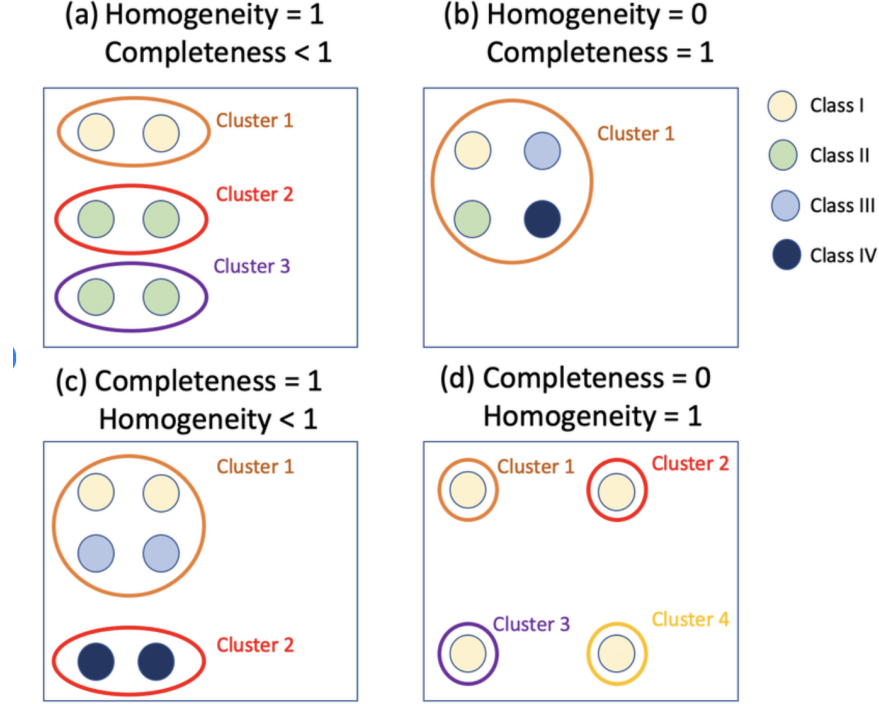


Figure 9: **Homogeneity versus completeness on toy clusterings.** Disc colour is the ground-truth class (I–IV, key at right); outlines are the predicted clusters. (a) clusters are pure but a class is split across several:  $h = 1$ ,  $c < 1$ . (b) a single cluster mixes every class:  $h = 0$ ,  $c = 1$ . (c) each class stays together but the clusters are impure:  $c = 1$ ,  $h < 1$ . (d) every protein isolated in its own cluster:  $h = 1$ ,  $c = 0$ . Reporting  $h$  and  $c$  separately distinguishes over-merging from over-splitting.

**Pair false-positive and false-negative rates.** At the level of protein *pairs*, a pair is a positive if the two proteins share a Foldseek reference fold and a negative otherwise; the learned clustering either co-clusters them or not. Counting agreements and disagreements over all  $\binom{n}{2}$  pairs,

$$\text{TP} = \sum_{c,k} \binom{n_{ck}}{2}, \quad \text{FP} = \sum_k \binom{n_k}{2} - \text{TP}, \quad \text{FN} = \sum_c \binom{n_c}{2} - \text{TP}, \quad \text{TN} = \binom{n}{2} - \text{TP} - \text{FP} - \text{FN},$$

where a *false positive* is a pair co-clustered by the model but lying in two different reference folds (an over-merge), and a *false negative* is a same-fold pair split apart (an over-split). We report the two rates

$$\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}} = \frac{\text{FP}}{\binom{n}{2} - \sum_c \binom{n_c}{2}}, \quad \text{FNR} = \frac{\text{FN}}{\text{FN} + \text{TP}} = \frac{\text{FN}}{\sum_c \binom{n_c}{2}}.$$

FPR is the fraction of cross-fold (negative) pairs the embedding wrongly glues together — the pair-level face of fusion — and FNR the fraction of same-fold (positive) pairs it fails to co-cluster — the pair-level face of fragmentation. A low FPR is the primary target, since merging unrelated folds is the failure mode we most want to avoid; read together, the two rates locate the model on the over-merge / over-split axis.

**TM- $\rho$ .** A clustering-free continuous check: over a sampled set  $\mathcal{P}$  of protein pairs with known TM-scores,

$$\text{TM-}\rho = \text{Spearman}_{(a,b) \in \mathcal{P}} \left( 1 - \mathbf{z}_a^\top \mathbf{z}_b, \text{TM}(a, b) \right),$$

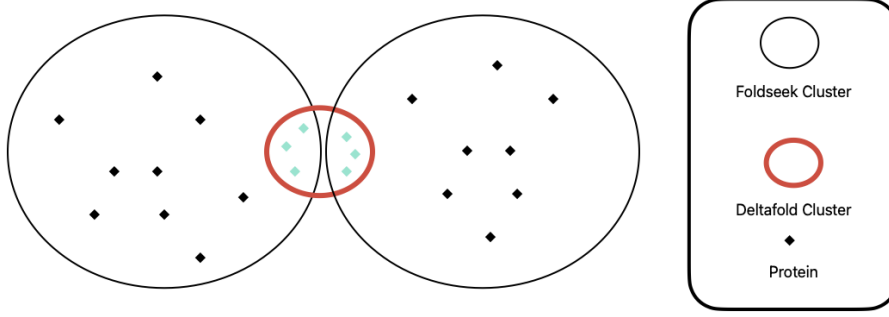


Figure 10: **False positives at the pair level.** Thin black ellipses are two distinct Foldseek reference folds; the red ellipse is one DeltaFold cluster that straddles both. Every pair of the highlighted proteins (green) that it co-clusters across the two folds is a false positive. The false-positive rate FPR counts these merged cross-fold pairs over all cross-fold pairs; it is the pair-level face of fusion (over-merging unrelated folds), the failure mode we most want to avoid.

the rank correlation between embedding cosine distance and structural similarity. A negative value is wanted (similar structures close together); it is a secondary sanity check rather than the primary target.

The practical consequence is that no single epoch is a reliable readout. A fairer summary is the level and trend of the homogeneity/completeness and fragmentation/fusion pair over the last several epochs, averaged over a few random seeds, gated by the health checks of Section 8.4.

## 8.2 Baselines

I want every number reported against two references on the same split: a random or shuffled-embedding baseline as a floor, and Foldseek [8] as the method to beat. A result only means something relative to both.

## 8.3 Design variables to sweep

The experiments are organised as one-factor-at-a-time sweeps around a base config, each run scored on the same cold split (Section 5.3) with the primary metrics of Section 8 (homogeneity/completeness and fragmentation/fusion) and gated by the health checks of Section 8.4. The central question is whether the message-passing topology — in particular the inter-SSE coupling — is what limits fold recovery, so the message-passing order is the first axis.

**Message-passing order (tensor diagram).** The main variable. Holding the model width, depth and objective fixed, we vary the `--tensor-diagram` (Section 6.5):

- *Topotein default* — the four-step diagram of Section 6.4 (edges with SSE context, SSE update through outer edges, residue refinement, protein), the reference.
- *Residue-hub control* — SSEs and edges communicate only through rank 0 (no  $N^{2 \rightarrow 1}$ ), the ablation that isolates the effect of the inter-SSE channel.
- *SSE-adjacency variant* — add a rank-2 adjacency  $A^{2 \sim 1}$  so SSEs exchange messages directly through shared contacts, on top of the outer edges.
- *Reordered sweeps* — permute the step order (e.g. update residues before SSEs, or SSEs before edges) to test the sensitivity to the bottom-up-then-down schedule.

- *No-rank-3 variant* — drop the protein channel and use a pooling readout, testing whether the global cell helps or over-fits.

The comparison *Topotein default vs. residue-hub control* is the decisive one: if fragmentation/fusion improve with the outer-edge channel, the earlier underperformance was topological, not geometric.

**Positive-pair construction.** The substructure fraction  $f \in [f_{lo}, f_{hi}]$  (Section 7.2) is the InfoMin knob and is swept in a small grid, e.g.  $[0.3, 0.5]$ ,  $[0.5, 0.8]$ ,  $[0.7, 0.9]$ , plus the sampling mode (contiguous chain segment vs. spatial ball) and the minimum substructure size. Too small and the two views stop sharing the fold; too large and they become near-identical.

**Objective and geometry.** Contrastive temperature  $\tau \in \{0.05, 0.1, 0.2\}$ ; number of interaction layers  $L \in \{3, 4, 6\}$ ; vector width  $d_v \in \{8, 16\}$ ; TCP bottleneck  $\lambda$ ; edge-centric vs. per-rank frames; and the readout strategy (node pooling vs. protein cell). Each is moved alone from the base config.

Table 1: One-factor-at-a-time sweep. Base config in bold; each run changes a single row.

| Axis               | Base                   | Swept values                                          |
|--------------------|------------------------|-------------------------------------------------------|
| Tensor diagram     | <b>Topotein 4-step</b> | residue-hub, $+A^{2 \sim 1}$ , reorderings, no-rank-3 |
| Substructure $f$   | $[0.5, 0.8]$           | $[0.3, 0.5]$ , $[0.7, 0.9]$                           |
| Substructure mode  | <b>contiguous</b>      | spatial ball                                          |
| Temperature $\tau$ | <b>0.2</b>             | 0.05, 0.1                                             |
| Layers $L$         | <b>4</b>               | 3, 6                                                  |
| Vector width $d_v$ | <b>16</b>              | 8                                                     |
| Readout            | <b>node pooling</b>    | protein cell                                          |

## 8.4 Health checks I want to run every epoch

Because a contrastive encoder can cheat by collapsing all embeddings toward a common direction, I want to log a few diagnostics every epoch and treat them as gating criteria rather than something I look at afterwards. These are the spread of the embedding values per dimension (it should stay clearly above the collapse threshold of about 0.02), the mean off-diagonal cosine similarity (it should stay low; values near 0.9 to 0.97 mean the vectors are nearly parallel), and the effective rank of the embedding covariance as a single number for how many dimensions are actually being used. I would only interpret a checkpoint biologically if these are healthy at that epoch.

## References

- [1] J. Nomburg, E. E. Doherty, N. Price, D. Bellieny-Rabelo, Y. K. Zhu, J. A. Doudna. Birth of protein folds and functions in the virome. *Nature*, 633:710–717, 2024. <https://doi.org/10.1038/s41586-024-07809-y>.
- [2] Z. Wang, A. R. Jamasb, et al. Topotein: Topological Deep Learning for Protein Representation Learning. *arXiv preprint* arXiv:2509.03885, 2025.
- [3] M. Hajij, G. Zamzmi, T. Papamarkou, N. Miolane, A. Guzmán-Sáenz, K. N. Ramamurthy, T. Birdal, T. K. Dey, S. Mukherjee, S. N. Samaga, N. Livesay, R. Walters, P. Rosen, M. T. Schaub. Topological Deep Learning: Going Beyond Graph Data. *arXiv preprint* arXiv:2206.00606, 2022.

- [4] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, G. E. Dahl. Neural Message Passing for Quantum Chemistry. *Proceedings of ICML*, 2017. arXiv:1704.01212.
- [5] T. Wang, P. Isola. Understanding Contrastive Representation Learning through Alignment and Uniformity on the Hypersphere. *Proceedings of ICML*, 2020. arXiv:2005.10242.
- [6] T. Chen, S. Kornblith, M. Norouzi, G. Hinton. A Simple Framework for Contrastive Learning of Visual Representations (SimCLR). *Proceedings of ICML*, 2020. arXiv:2002.05709.
- [7] A. van den Oord, Y. Li, O. Vinyals. Representation Learning with Contrastive Predictive Coding (InfoNCE). *arXiv preprint* arXiv:1807.03748, 2018.
- [8] M. van Kempen, S. S. Kim, C. Tumescheit, M. Mirdita, J. Lee, C. L. M. Gilchrist, J. Söding, M. Steinegger. Fast and accurate protein structure search with Foldseek. *Nature Biotechnology*, 42:243–246, 2024.
- [9] R. J. G. B. Campello, D. Moulavi, J. Sander. Density-Based Clustering Based on Hierarchical Density Estimates (HDBSCAN). *Advances in Knowledge Discovery and Data Mining (PAKDD)*, 2013.
- [10] B. Jing, S. Eismann, P. Suriana, R. J. L. Townshend, R. Dror. Learning from Protein Structure with Geometric Vector Perceptrons. *Proceedings of ICLR*, 2021. arXiv:2009.01411.
- [11] A. Morehead, J. Cheng. Geometry-Complete Perceptron Networks for 3D Molecular Graphs. *Bioinformatics*, 2024. arXiv:2211.02504.
- [12] R. Bro, E. Acar, T. G. Kolda. Resolving the Sign Ambiguity in the Singular Value Decomposition. *Journal of Chemometrics*, 22:135–140, 2008.
- [13] K. T. Schütt, P.-J. Kindermans, H. E. Sauceda, S. Chmiela, A. Tkatchenko, K.-R. Müller. SchNet: A Continuous-Filter Convolutional Neural Network for Modeling Quantum Interactions. *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. arXiv:1706.08566.
- [14] K. He, H. Fan, Y. Wu, S. Xie, R. Girshick. Momentum Contrast for Unsupervised Visual Representation Learning (MoCo). *Proceedings of CVPR*, 2020. arXiv:1911.05722.
- [15] P. Hermosilla, T. Ropinski. Contrastive Representation Learning for 3D Protein Structures. *arXiv preprint* arXiv:2205.15675, 2022.
- [16] Z. Zhang, M. Xu, A. Jamasb, V. Chenthamarakshan, A. Lozano, P. Das, J. Tang. Protein Representation Learning by Geometric Structure Pretraining (GearNet). *Proceedings of ICLR*, 2023. arXiv:2203.06125.
- [17] L. Jing, P. Vincent, Y. LeCun, Y. Tian. Understanding Dimensional Collapse in Contrastive Self-Supervised Learning. *Proceedings of ICLR*, 2022. arXiv:2110.09348.
- [18] A. Rosenberg, J. Hirschberg. V-Measure: A Conditional Entropy-Based External Cluster Evaluation Measure. *Proceedings of EMNLP-CoNLL*, 2007.